APACHE KAFKA — ARCHITECTURE & CONFIGURATION

by Jyothi Prakash Kalmanje V2

How it works, and how to configure it for each workload

Defaults for Kafka 3.x. green = default 3.0 = default changed in 3.0.
KRaft is the production control plane from 3.3; ZooKeeper deprecated from 3.5, removed in 4.0. Verify against your broker version.

1 · WHAT KAFKA IS, AND WHY IT EXISTS

start here — the problem it was built for, before the configuration

The definition

Kafka is a **distributed, append-only, replicated commit log**. Producers append records to the end of a log; consumers read forward from a position they choose and remember. Nothing is removed when it is read.

The one distinction that matters: a **queue** delivers a message, gets an acknowledgement, and **deletes it**. A **log** appends records and **keeps them** for a retention period, while each consumer tracks its own offset. Almost every design mistake with Kafka comes from treating it as a queue.

What follows from being a log

- **Many independent consumers** read the same records without competing — each has its own offset.
- **Replay is ordinary.** Reset an offset and reprocess history; you are not recovering a deleted message.
- **Order is per partition**, never across a topic. This is the constraint everything else is designed around.
- **Consumers can be slow** or offline without blocking producers — the log is the buffer.

Why it is fast

- **Sequential writes** to an append-only file, not random updates.
- **Batching and compression** across many records rather than per message.
- **Consumers pull**, so a slow consumer never overwhelms itself.
- **Partitions parallelise** — throughput scales by adding partitions and consumers together.

The problem it solves

Before a log, systems integrated point to point. Every new consumer of an event meant a new connection into the producer, and each producer carried the delivery concerns of everyone downstream.

Without a log	With a log
N×M integrations — every producer wired to every consumer	Producers write once; any number of consumers subscribe independently
Producer knows its consumers, and changes when they change	Producers publish facts and do not know who reads them
A slow consumer applies backpressure to the producer, or drops data	The log absorbs the difference; the consumer catches up later
No replay — a bug means the data is gone	Reset the offset and reprocess from retained history
A new consumer needs a backfill project	A new consumer starts at the beginning of retention
Bursts must be handled live	Bursts are written to disk and drained at a sustainable rate

Kafka is infrastructure for decoupling, not just messaging. The durability and replay are what let teams evolve producers and consumers on separate schedules — which is usually the real reason it is adopted.

The vocabulary, in one place

Topic	A named stream of records
Partition	An ordered, append-only shard of a topic. The unit of parallelism *and* of ordering
Offset	A record's position in a partition. Consumers store it; the broker does not track per-consumer state
Producer	Appends records, choosing a partition by key
Consumer group	Consumers sharing the work of a topic; each partition goes to exactly one member
Broker	A server holding partitions
Replica / ISR	Copies of a partition; the in-sync set is what durability is defined against
Retention	How long records are kept — by time, size, or compaction to the latest value per key

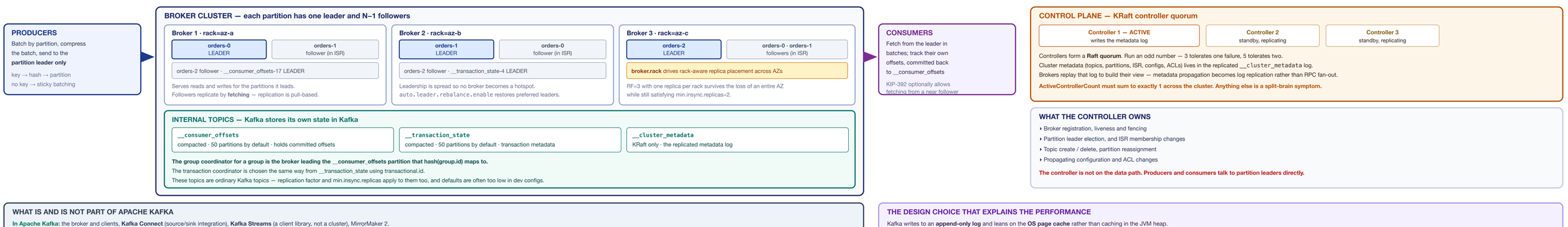
When it fits, and when it does not

Use it for	Use something else for
Event streaming and integration between services	**Task queues** with per-message ack and retry — a queue is simpler
Activity, metrics and log pipelines	**Request/reply** — use RPC, not a log
Change data capture and replication	**Low-volume messaging** — the operational cost is not repaid
Stream processing and materialised views	**Strict global ordering** — it does not exist across partitions
Event sourcing and audit trails	**Per-message priority or delay** — not what a log does

Kafka is not free to run. Brokers, storage, partition planning, consumer lag monitoring and upgrades are ongoing work. Adopt it when decoupling, replay or throughput genuinely justify that — not because it is the default answer to "we need messaging".

2 · CLUSTER ARCHITECTURE

brokers, the controller quorum, partitions and the internal topics that make it work



WHAT IS AND IS NOT PART OF APACHE KAFKA

In Apache Kafka: the broker and clients, **Kafka Connect** (source/sink integration), **Kafka Streams** (a client library, not a cluster), MirrorMaker 2.

Not in Apache Kafka: Schema Registry, ksqlDB, Control Center, and the REST Proxy are Confluent products with their own licences.

Kafka Streams runs inside your application process — there is no separate cluster to operate, but its state stores and internal topics are yours to manage.

THE DESIGN CHOICE THAT EXPLAINS THE PERFORMANCE

Kafka writes to an **append-only log** and leans on the **OS page cache** rather than caching in the JVM heap.

Consumers reading recent data are served from page cache, so throughput comes from sequential I/O, not from RAM sized in the broker.

This is why brokers run a modest heap and leave most system memory to the OS — a large Kafka heap is usually a misconfiguration.

THE FIVE INVARIANTS EVERYTHING ELSE FOLLOWS FROM

1 · **Ordering exists within a partition only.** Never across partitions, never across topics.
2 · **All writes and reads for a partition go through its leader** — followers replicate by fetching; they do not serve writes.
3 · **The log is immutable and append-only.** Compaction and retention delete whole records or segments; nothing is edited in place.
4 · **Consumers own their position.** The broker tracks no per-consumer cursor — offsets are committed data.
5 · **Consumers read only up to the high watermark** — data not yet on all in-sync replicas is invisible to them.

Almost every Kafka surprise in production traces back to one of these five being assumed away.

3 · THE LOG — STORAGE MODEL

topic → partition → segment → record batch

A partition is a sequence of segment files. Only the active segment is written to.



Retention and compaction only ever act on closed segments. A low-traffic topic with a large segment size can hold data far past retention.ms.
Roll the active segment sooner with `segment.ms` when prompt expiry matters — regulatory deletion, for instance.

Offsets and the three markers that govern visibility



Records between HW and LEO exist on the leader but are invisible to consumers — they are not yet safe, because the leader could fail before followers catch up.
The offset is a position in a partition, permanently. It is never reused, and it is not a global sequence — offset 104 in partition 0 has no relation to offset 104 in partition 1.

Retention vs compaction — two different jobs

cleanup.policy=delete — a time-bounded buffer

Whole segments are removed once every record in them is older than `retention.ms`, or the partition exceeds `retention.bytes`.

Whichever limit is reached first triggers deletion.

Use for: event streams, logs, metrics — anything where old data stops mattering.

Consumers that fall behind retention lose data permanently and will reset according to `auto.offset.reset`.

cleanup.policy=compact — a keyed snapshot

Retains at least the **latest value for every key**, forever. Older values for the same key are removed by the log cleaner.

A record with a **null value is a tombstone** — it marks the key deleted and is itself removed after `delete.retention.ms`.

Use for: changelogs, CDC, current-state tables, configuration.

A consumer offline longer than delete.retention.ms can miss a deletion entirely and keep stale state.

4 · REPLICATION & ISR

how durability is actually achieved

The mechanism

- Each partition has one **leader** and N−1 **followers**, where N = replication factor
- **Replication is pull-based** — followers issue fetch requests to the leader exactly as consumers do. The leader does not push
- A follower is **in-sync** if it has fetched up to the leader within `replica.lag.time.max.ms` `30000`
- The **ISR** is the set of in-sync replicas, including the leader. It shrinks and grows dynamically
- The **high watermark** advances to the lowest log end offset across the ISR — that is what makes a record readable

This is why acks=all and min.insync.replicas are one mechanism, not two settings. `acks=all` waits for the ISR; `min.insync.replicas` defines how small the ISR may become before writes are rejected. Without the second, the first can be satisfied by a single-member ISR.

Leader election

Situation	Behaviour
Leader fails, ISR non-empty	Controller promotes an ISR member. No data loss
Leader fails, ISR empty, `unclean=false` `default`	Partition goes **offline** until an ISR member returns. Consistency over availability
Leader fails, ISR empty, `unclean=true`	An out-of-sync replica is promoted. **Committed records are silently lost**
Broker returns	`auto.leader.rebalance.enable` `true` restores preferred leaders to rebalance load

Leader epoch

Every leadership change increments a monotonic epoch number (KIP-101). Followers use it to determine exactly where their log diverged from the new leader and truncate correctly. It exists because truncating on the high watermark alone could produce divergent logs across replicas during rapid leadership changes.

What to watch

Metric	Meaning
`UnderReplicatedPartitions`	ISR smaller than RF — durability degraded now
`UnderMinIsrPartitions`	ISR below min.insync.replicas — writes rejected
`OfflinePartitionsCount`	No leader — data unavailable
`IsrShrinksPerSec`	Frequent shrink/expand means an overloaded or lagging broker

UnderReplicatedPartitions is the single most important broker alert. A sustained non-zero value means you are running with less redundancy than you designed for — and you will find out how much less during the next failure.

5 · WRITE PATH & READ PATH

what actually happens on send() and poll()

Write — producer to acknowledged

```
send() appends to an in-memory batch, per partition
  ← returns immediately with a Future
batch closes on batch.size or linger.ms, whichever first
compression applied to the whole batch
sent to the partition leader only
leader appends to its log → OS page cache
  ← not fsync'd per record by default
followers fetch and append
high watermark advances
ack returned per acks setting
```

Kafka does not fsync every record. Durability comes from **replication across brokers**, not from flushing to disk on each write. This is deliberate — it is why `acks` and replication factor matter far more than any flush setting, and why `log.flush.interval.messages` is best left alone.

Compression is per batch, not per record. So `batch.size` and `linger.ms` directly determine the compression ratio you achieve. Fuller batches compress better and cost fewer requests.

Idempotent producer internals

- Producer is assigned a **producer ID** and an **epoch**
- Each record batch carries a **sequence number per partition**
- The broker rejects a duplicate or out-of-order sequence, so a retried batch is written once
- This is why `max.in.flight` ≤ 5 is required — the broker tracks a bounded sequence window

Read — poll to record

```
consumer sends a fetch to the partition leader
  ← or a near replica if KIP-392 is configured
broker waits for fetch.min.bytes
  or until fetch.max.wait.ms elapses
data is served from the OS page cache
  when it is recent — sequential, no disk seek
batches are returned still compressed
  ← the consumer decompresses
consumer reads up to the high watermark
  ← or the LSO under read_committed
offsets committed to __consumer_offsets
```

Zero-copy, and when you lose it. For plaintext connections Kafka can transfer bytes from the page cache to the socket without copying them through the JVM. **Enabling TLS removes that optimisation**, because the data must be encrypted in user space. TLS is still the right choice in production — but budget for the CPU cost rather than being surprised by it.

The consumer, not the broker, holds the cursor. Nothing in the broker tracks where a given consumer is. That is why replay is trivial — seek to an offset or timestamp and read again — and why an accidental offset reset silently reprocesses or skips data.

Fetch sizing

Parameter	Raise it to
`fetch.min.bytes`	Batch more per request, cut broker load, add latency
`fetch.max.wait.ms`	Bound the latency cost of the above
`max.partition.fetch.bytes`	Improve throughput per partition

6 · CONSUMER GROUPS

scaling and coordination

The assignment rule

- Each partition is consumed by **exactly one consumer within a group**
- A consumer may hold many partitions; a partition is never split across consumers in the same group
- **Partition count caps parallelism.** Consumers beyond the partition count sit idle
- Different groups consume the same topic independently, each with its own offsets

Who does what during a rebalance

Role	Responsibility
Group coordinator a broker	Tracks membership and liveness, and stores committed offsets
Group leader one of the consumers	**Computes the partition assignment** and returns it via the coordinator

Assignment is computed client-side. The broker coordinates membership but does not decide who gets which partition — the elected consumer does, using the configured assignor. That is why the assignment strategy is a *client* setting and why all members of a group must agree on it.

Offset storage

Committed offsets are records in __consumer_offsets, a compacted internal topic keyed by (group, topic, partition). Compaction keeps the latest commit per key, so the topic stays bounded no matter how long the group runs.

Delivery semantics follow from commit order

Order	Result
Commit **before** processing	**At-most-once** — crash loses the record
Commit **after** processing	**At-least-once** — crash reprocesses it
Commit **inside a transaction**	**Exactly-once** within Kafka

Auto-commit gives you neither reliably. It commits on a timer, independent of whether your processing finished — so a crash can lose records the group believes were consumed. Disable it wherever loss is unacceptable and commit explicitly after the work succeeds.

7 · THE DURABILITY MODEL

three settings decide whether you can lose data — and they only work together

acks — how many replicas must confirm

`acks=0` fire and forget — silent loss on any failure. Metrics only
The producer does not even wait for a network-level response.

`acks=1` leader confirms — loses data if the leader fails before replication
Acceptable for telemetry where a dropped sample costs nothing.

`acks=all (-1)` all *in-sync* replicas confirm `DEFAULT SINCE 3.0`
No loss while min.insync.replicas remain. Costs one replication round trip.

acks=all alone is not enough — "all in-sync replicas" can mean one replica.
It is a floor on *who confirms*, not on *how many exist*. The floor is set by min.insync.replicas.

min.insync.replicas — the ISR floor

`=1` acks=all silently degrades to acks=1 whenever followers lag

`=2` With RF=3: survives one broker loss and remains writable

`=3` With RF=3: any single broker loss stops all writes

Set min.insync.replicas = RF − 1
That tolerates one failure while staying writable. Setting it equal to RF trades all availability for no additional durability.

A rejected write is NotEnoughReplicasException — the producer sees it and can retry.

What each combination gives you

RF · min.isr · acks	Broker losses tolerated	Verdict
1 · 1 · 1	none	Dev only
3 · 1 · all	2 (writes) — but data at risk	False safety
3 · 2 · all	1	**Production standard**
3 · 3 · all	0 — writes stop	Over-tight
5 · 3 · all	2	Critical / multi-AZ

`broker.rack` spreads replicas across AZs. Separately, `replica.selector.class` (KIP-392) lets consumers fetch from a near replica to cut cross-AZ egress.

IDEMPOTENCE & ORDERING

`enable.idempotence=true` is default from 3.0 and requires acks=all, retries>0, and max.in.flight ≤ 5 — the client enforces this.
It removes duplicates caused by **producer retries** only. It does not make a multi-step pipeline exactly-once; that needs transactions.
Ordering is guaranteed **within a partition**. Same key ⇒ same partition ⇒ ordered. With idempotence off and max.in.flight>1, a retry can reorder.
Adding partitions later changes the key→partition mapping and breaks ordering for existing keys. Partition count cannot be reduced. Size it up front.
Choose the partition key to match the ordering requirement — and check its cardinality, or one key becomes a hot partition.

TIMEOUT HIERARCHY — set these consistently or the client behaves unpredictably

`delivery.timeout.ms` `120000` — total budget from send() to success or failure. Must be ≥ request.timeout.ms + linger.ms.
`request.timeout.ms` `30000` — wait for one broker response before retrying.
`max.block.ms` `60000` — how long send() blocks when the buffer is full or metadata is missing.
retries defaults to Integer.MAX_VALUE, so the retry count bounds nothing. delivery.timeout.ms is the real deadline — tune that.
On the consumer side the equivalent trap is max.poll.interval.ms — see panel 10.

8 · PRODUCER CONFIG

client-side

Parameter	Values	Effect
`acks`	`0` · `1` · `all` `3.0`	Durability vs latency
`enable.idempotence`	`true` `3.0` · false	De-duplicates producer retries
`compression.type`	`none` · gzip · snappy · lz4 · zstd	Applied per batch — interacts with batch.size
`batch.size`	`16384` bytes	Max bytes per partition batch
`linger.ms`	`0` · 5–100 typical	Wait to fill a batch. **Biggest throughput lever**
`buffer.memory`	`33554432` (32 MB)	Client send buffer; exhaustion blocks up to max.block.ms
`max.in.flight.requests.per.connection`	`5` · 1	>5 rejected with idempotence enabled
`retries`	`2147483647`	Effectively infinite; bounded by delivery.timeout.ms
`delivery.timeout.ms`	`120000`	The real end-to-end send deadline
`max.request.size`	`1048576` (1 MB)	Must align with topic max.message.bytes
`transactional.id`	`null` · unique string	Enables transactions; implies idempotence
`partitioner.class`	`null`	Built-in partitioning; sticky batching for null keys

linger.ms=0 does not minimise latency under load. It produces many small requests, which increases queueing and broker work. A small linger often *reduces* end-to-end latency at high throughput while improving compression. `TUNE`

Compression choice. `lz4` and `snappy` favour speed; `zstd` gives the best ratio at higher CPU. Set it on the producer; the topic-level default `producer` means "keep whatever the producer sent" and avoids a broker-side recompression cost.

9 · CONSUMER CONFIG

client-side

Parameter	Values	Effect
`enable.auto.commit`	`true` · false	Commits on a timer, not on completion
`auto.commit.interval.ms`	`5000`	How often the timer fires
`auto.offset.reset`	earliest · `latest` · none	Only used with no committed offset. **latest skips backlog**
`isolation.level`	`read_uncommitted` · read_committed	read_committed reads only to the LSO
`max.poll.records`	`500`	Main control on processing time per loop
`max.poll.interval.ms`	`300000` (5 min)	Exceed it and the consumer leaves the group
`session.timeout.ms`	`45000` `3.0`	Heartbeat liveness window
`heartbeat.interval.ms`	`3000`	Keep near one third of session.timeout.ms
`fetch.min.bytes`	`1`	Raise to batch fetches, cut broker load
`fetch.max.wait.ms`	`500`	Latency bound on fetch.min.bytes
`max.partition.fetch.bytes`	`1048576`	**Soft limit** — an oversized first batch is still returned, so the consumer cannot stall (KIP-74)
`partition.assignment.strategy`	`[Range, CooperativeSticky]` `3.0` · RoundRobin · Sticky	Cooperative avoids stop-the-world rebalances
`group.instance.id`	`null` · unique string	Static membership — no rebalance on brief restart

The default that causes the most data-integrity bugs. Auto-commit advances offsets on a timer regardless of whether processing finished. A crash between the commit and completion loses those records permanently — Kafka believes they were consumed.

10 · TOPIC & BROKER CONFIG

server-side

Parameter	Values	Effect
`cleanup.policy`	`delete` · compact · compact,delete	Time/size deletion vs latest-per-key
`retention.ms`	`604800000` (7d) · `-1` = never	Ignored under pure compaction
`retention.bytes`	`-1`	Per-partition cap; first limit hit wins
`segment.bytes`	`1073741824` (1 GB)	Deletion and compaction act on closed segments only
`segment.ms`	`604800000` (7d)	Force a roll by time — lower for prompt expiry
`min.insync.replicas`	`1` · **2 in production**	Only meaningful with acks=all
`unclean.leader.election.enable`	`false` · true	true permits data loss to regain availability
`max.message.bytes`	`1048588`	Exceeded ⇒ broker rejects with RecordTooLargeException
`min.cleanable.dirty.ratio`	`0.5`	How dirty before the cleaner runs
`delete.retention.ms`	`86400000` (24h)	Tombstone lifetime — consumers must read within it
`num.partitions`	`1`	Default for auto-created topics — set explicitly
`replica.lag.time.max.ms`	`30000`	Lag before a follower leaves the ISR
`message.timestamp.type`	`CreateTime` · LogAppendTime	Producer clock vs broker clock
`auto.create.topics.enable`	`true` · **false in prod**	true turns typos into permanent topics

Internal topics need production settings too. `offsets.topic.replication.factor` and `transaction.state.log.replication.factor` default to 3, but single-broker dev setups often lower them — and that value persists if the topic was created before the cluster grew.

11 · CONFIGURATION PROFILES BY USE CASE

the same parameters set six ways — start from the closest column, then load-test

Parameter	A · FINANCIAL orders, payments — no loss	B · HIGH THROUGHPUT logs, metrics, IoT telemetry	C · LOW LATENCY alerts, real-time decisioning	D · EXACTLY-ONCE stream processing pipelines	E · COMPACTED STATE CDC, changelog, config	F · EVENT STORE replay, audit, sourcing
acks	all	1	1	all	all	all
enable.idempotence	true	true	true	true *(implied)*	true	true
compression.type	lz4	zstd	none / lz4	lz4	zstd	zstd
batch.size	32768	262144+	16384	32768	65536	131072
linger.ms	5–10	50–100	0–2	5–10	20–50	50–100
transactional.id	optional	–	–	**required**	optional	–
replication.factor	**3**	2–3	3	**3**	**3**	**3**
min.insync.replicas	**2**	1	2	**2**	**2**	**2**
unclean.leader.election	false	true *(tolerable)*	false	false	false	false
cleanup.policy	delete	delete	delete	delete	**compact**	delete *(long)*
retention.ms	7–30 d	1–7 d	hours–1 d	7 d	n/a *(compact)*	–1 *or years*
segment.ms / bytes	default	large	small	default	small *(compaction)*	large
enable.auto.commit	**false**	true	true	**false** *(EOS)*	**false**	**false**
isolation.level	read_committed	read_uncommitted	read_uncommitted	**read_committed**	read_committed	read_committed
auto.offset.reset	earliest	latest	latest	earliest	earliest	earliest
max.poll.records	100–500	500–2000	1–50	100–500	500	500–2000
processing.guarantee Kafka Streams	at_least_once	at_least_once	at_least_once	**exactly_once_v2**	exactly_once_v2	at_least_once
Partition key	entity id *(ordering)*	null *(spread)*	entity id	join key	**primary key**	aggregate id

Read the columns, not the cells. Column A works only because acks, min.insync.replicas, RF, unclean election and manual commit are all aligned. Change one in isolation and the guarantee disappears without any error.

Starting points, not answers. Batch, linger and poll values depend on message size, throughput and hardware. Take the nearest column, load-test with *your* payloads, then change one variable at a time. `TUNE`

Column B accepts loss deliberately. acks=1, min.isr=1 and unclean election are right for telemetry where a missing sample costs nothing. Write that reasoning down — otherwise it gets copied into a workload where it is wrong.

12 · GROUP LIVENESS

where consumers actually break

Two independent liveness checks

Mechanism	Fails when
`session.timeout.ms` background heartbeat thread	Process dies or the network partitions — **heartbeats stop**
`max.poll.interval.ms` the poll loop	Process alive but **stuck processing**; poll() not called again in time

The classic production failure. Processing slows, one poll cycle exceeds `max.poll.interval.ms`, the consumer is evicted, the group rebalances, partitions move to another instance that is also slow, and it repeats. The group spends its time rebalancing instead of consuming while lag grows — and no component reports an error.

The fix, in order of preference

- **Reduce `max.poll.records`** so each loop finishes well inside the interval
- **Move slow work off the poll thread** to a bounded worker pool, with pause/resume for backpressure
- **Raise `max.poll.interval.ms`** only once you know the real processing-time distribution — it delays detection of genuinely stuck consumers
- **Adopt `CooperativeStickyAssignor`** so rebalances stop being stop-the-world
- **Set `group.instance.id`** so restarts inside the session window skip rebalancing

Eager vs cooperative

Protocol	Behaviour
Eager Range, RoundRobin	Every consumer revokes *all* partitions, then reassigns — the whole group stops
Cooperative CooperativeSticky	Only partitions that must move are revoked; the rest keep consuming

Migrating between protocols needs a documented two-step rolling upgrade — the group must not contain members using incompatible protocols simultaneously. Check the procedure for your client version rather than switching in place.

13 · EXACTLY-ONCE

scope and cost

Level	Guarantee
At-most-once	Commit before processing. Loss on crash, never duplicates
At-least-once the common default	Commit after processing. Duplicates on crash, never loss
Idempotent producer	Removes duplicates from *producer retries* only
Transactional	Atomic across consume → process → produce, including offsets

What a transaction spans

```
// the atomic unit: output records AND the offsets
producer.initTransactions()
producer.beginTransaction()
  producer.send(outputRecord)          // 1..n topics
  producer.sendOffsetsToTransaction(   // consumed pos.
      offsets, consumerGroupMetadata)
producer.commitTransaction()
// consumers must set isolation.level=read_committed
```

Committing offsets inside the transaction is the whole mechanism. Output and consumption progress succeed or fail together, closing the gap that makes at-least-once produce duplicates. The coordinator writes commit or abort markers into the affected partitions.

The boundary that gets misunderstood

Exactly-once applies within Kafka. It covers Kafka topics in, processing, and Kafka topics out — atomically. It does **not** extend to an external database, a payment API, or an email send. Those need idempotency keys on your side.

Costs

- Extra coordination round trips — throughput falls relative to at-least-once
- Consumers see records only at commit boundaries, so **latency rises with transaction size**
- An open transaction holds the LSO back, stalling `read_committed` consumers
- Kafka Streams under `exactly_once_v2` commits far more frequently by default, trading throughput for shorter latency

Ask whether you need it. Many pipelines are simpler and cheaper with at-least-once plus a consumer made idempotent on a business key.

14 · MISCONFIGURATIONS THAT REACH PRODUCTION

each individually reasonable — wrong only in combination, and Kafka does not warn you

Configuration	What actually happens
acks=all with min.isr=1	Reads as maximum durability; degrades to acks=1 whenever followers lag. Loss with no error
min.isr = RF	Any single broker restart stops all writes to the topic
Auto-commit with slow processing	Offsets advance past records never processed. Silent loss
auto.offset.reset=latest on a new consumer	Skips the entire existing backlog on first start — and looks like it worked
Adding partitions to a keyed topic	Key→partition mapping changes; ordering breaks for existing keys
Partition count sized to today	Cannot be reduced; growing it later breaks keyed ordering
producer max.request.size > topic max.message.bytes	Broker rejects with RecordTooLargeException — surfaces at the producer, long after the topic was set up
Tuning retries instead of delivery.timeout.ms	retries is effectively infinite by default; the timeout is the real deadline

Configuration	What actually happens
Compaction without tombstone awareness	Consumers offline beyond delete.retention.ms miss deletions and keep stale state
Large segments on a low-volume topic	Data retained far past retention.ms; regulatory deletion silently misses
Internal topics left at dev settings	__consumer_offsets at RF=1 — a broker loss loses committed offsets
One giant consumer group	Every deployment triggers a full rebalance under an eager assignor
Consumers > partitions	Surplus consumers idle; partition count caps parallelism
linger.ms=0 chasing latency	Tiny requests, more round trips, worse compression — often higher latency at load
Large JVM heap on brokers	Starves the page cache Kafka actually depends on
PLAINTEXT "for now"	Unknown clients accumulate; retrofitting auth becomes a migration

The two irreversible decisions. Partition count — increasable but never decreasable, and increasing it breaks keyed ordering. **Partition key** — determines ordering and downstream skew; changing it means reprocessing. Everything else on this sheet can be changed later. These two deserve a design review.

15 · SECURITY, QUOTAS & SIZING

access and capacity

Parameter	Values
`security.protocol`	`PLAINTEXT` · SSL · SASL_PLAINTEXT · **SASL_SSL**
`sasl.mechanism`	PLAIN · SCRAM-SHA-256 · **SCRAM-SHA-512** · GSSAPI · OAUTHBEARER
`ssl.endpoint.identification.algorithm`	`https` — empty disables hostname verification. **Do not.**
`ssl.client.auth`	`none` · requested · required (mTLS)

PLAINTEXT is the default: no encryption, no authentication. Any client that can reach the port can read and write every topic. Note also that **TLS disables the zero-copy path**, so budget CPU for it — but do enable it.

ACLs and quotas

- Producers need **Write** on the topic; transactional producers also need Write on the **TransactionalId**
- Consumers need **Read** on the topic **and** on the consumer group
- **`producer_byte_rate` / `consumer_byte_rate` / `request_percentage` throttle rather than reject** — a misbehaving client slows down instead of taking the cluster with it

Sizing

```
# partitions: satisfy both bounds
partitions ≥ throughput / per_partition_rate
partitions ≥ peak_consumer_instances

# storage
bytes ≈ rate × avg_size × retention_s × RF
                ÷ compression_ratio
# measure per-partition rates on YOUR hardware
```

More partitions is not free. Each costs file handles, memory on brokers and clients, and controller work. Over-partitioning raises latency and lengthens recovery after a broker failure. `TUNE`

Alert on lag *trend*, not a fixed threshold. Steady lag under load is normal; monotonically growing lag means consumption is losing to production and will not recover on its own.

16 · PRE-PRODUCTION CHECKLIST

confirm before a topic carries traffic you cannot afford to lose

- ☐ **Durability triple coherent** — RF, min.insync.replicas and acks match the stated guarantee
- ☐ **min.insync.replicas = RF − 1** unless a different trade is documented
- ☐ **unclean.leader.election.enable=false** on every topic that matters
- ☐ **Replicas spread by broker.rack** across racks or AZs
- ☐ **Internal topics at production RF** — __consumer_offsets and __transaction_state
- ☐ **Controller quorum is an odd number** (KRaft), 3 or 5
- ☐ **Partition count sized with headroom** — it cannot be reduced
- ☐ **Partition key chosen for ordering and skew**, and reviewed
- ☐ **Key cardinality checked** — no hot partition under real traffic
- ☐ **enable.idempotence=true** confirmed, not assumed from the version
- ☐ **delivery.timeout.ms tuned**, not the retry count
- ☐ **Message size limits aligned** producer ↔ topic
- ☐ **Auto-commit disabled** wherever loss is unacceptable
- ☐ **Offsets committed after processing succeeds**
- ☐ **auto.offset.reset deliberate** — latest silently skips backlog
- ☐ **max.poll.records fits real processing time**
- ☐ **CooperativeStickyAssignor adopted** for large groups
- ☐ **group.instance.id set** for stateful consumers
- ☐ **Retention verified by observation**, not just configuration
- ☐ **segment.ms lowered** where prompt expiry is required
- ☐ **Tombstone retention exceeds** the longest expected consumer outage
- ☐ **isolation.level=read_committed** wherever transactions are used
- ☐ **Exactly-once boundary understood** — external side effects not covered
- ☐ **SASL_SSL and ACLs in place** before the first production client
- ☐ **Hostname verification left enabled**
- ☐ **auto.create.topics.enable=false** on production brokers
- ☐ **Broker heap modest**, page cache left to the OS
- ☐ **Quotas set** on shared clusters
- ☐ **URP, UnderMinIsr, offline partitions and lag alerted**
- ☐ **ActiveControllerCount alerted** — must equal 1 cluster-wide
- ☐ **Disk headroom monitored** — a full disk takes the broker down
- ☐ **Broker failure rehearsed** — kill one and watch the guarantee hold

The test that validates the whole sheet. Take one broker out under production-like load. Writes should continue, no data should be lost, consumer lag should recover, and no alert should be a surprise. **Configuration never tested against a real failure is a hypothesis** — and the failure is the wrong moment to find out which setting you got wrong.